INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Introducción al Desarrollo de Nuevas Plataformas				
TÍTULO DE LA PRÁCTICA:	Persistencia de datos con Room Database en Jetpack Compose				
NÚMERO DE LA PRÁCTICA:	09	**AÑO LECTIVO:**	2026B	**NRO. SEMESTRE:**	VIII
FECHA DE PRESENTACIÓN:	31/10/2026	**HORA DE PRESENTACIÓN:**	23:59:00		
INTEGRANTE(s): - Jorge Luis Mamani Huarsaya - Yordano Hernan Boza Portilla				**NOTA (0 - 20):**	Nota colocada por el docente
DOCENTE: Roxana Evelyn Limache Calatayud					

RESULTADOS DE LOS EJERCICIOS PROPUESTOS

Link del repositorio GitHub: https://github.com/jorghee/introduction_new_platforms/tree/main/labs/lab09

Desarrollo de la Solución

Organización Modular del Código y Separación de Responsabilidades

En el desarrollo de aplicaciones móviles con Android y Jetpack Compose, concentrar todas las entidades, interfaces de acceso a datos, lógica de presentación y componentes de interfaz gráfica dentro de un único archivo (como `MainActivity.kt`) representa un antipatrón arquitectónico conocido como *God Object* o *Activity monolítica*. Este enfoque dificulta el mantenimiento, la depuración, las pruebas unitarias y la escalabilidad del sistema a medida que los requisitos crecen.

Por ello, de forma previa a la resolución de los ejercicios propuestos, se estructuró el proyecto aplicando el principio de responsabilidad única (*Single Responsibility Principle* - SRP) y la arquitectura recomendada por Google (MVVM - *Model-View-ViewModel* con patrón Repositorio), organizando los módulos en paquetes específicos dentro de `com.example.persistenciaroom_compose`:

Capa / Paquete	Componentes Clave	Responsabilidad Específica en la Solución
`data.local.entity`	`EdificioEntity`, `AmbienteEntity`, `NotaEntity`	Definen el esquema relacional de la base de datos SQLite. Cada clase representa una tabla física con sus claves primarias, foráneas e índices.
`data.local.relation`	`EdificioConAmbientes`, `AmbienteConNotas`	Estructuras de datos que agrupan entidades relacionadas mediante `@Embedded` y `@Relation` para modelar consultas relacionales 1:N.
`data.local.dao`	`EdificioDao`	Interfaz de acceso a datos (DAO). Declara de forma tipada las operaciones CRUD (`@Insert`, `@Update`, `@Delete`) y las consultas transaccionales (`@Transaction`).
`data.local`	`AppDatabase`	Clase abstracta que hereda de `RoomDatabase`. Centraliza la configuración de SQLite, versionamiento y patrón Singleton de acceso seguro.

Capa / Paquete	Componentes Clave	Responsabilidad Específica en la Solución
`data.repository`	`EdificioRepository`	Fuente única de verdad (*Single Source of Truth*). Abstrae la base de datos de las capas superiores y gestiona las operaciones asíncronas.
`ui.viewmodel`	`EdificiosViewModel`, `EdificiosViewModelFactory`	Mantiene el estado de la UI desacoplado del ciclo de vida de la Activity. Expone flujos reactivos `StateFlow` y procesa eventos de usuario.
`ui.components`	`EdificioItem`, `AmbienteNotaItem`, `DialogosCrud`	Componentes Composables modulares y reutilizables para tarjetas visuales, listas anidadas y ventanas modales de edición.
`ui.screens`	`EdificiosScreen`	Pantalla principal con navegación por pestañas (`TabRow`) y botón flotante de acción (`FloatingActionButton`).
`com.example...`	`MainActivity`	Punto de entrada de la aplicación. Se limita a inicializar el repositorio e inyectarlo en el ViewModel dentro de `setContent`.

Esta separación desacopla la persistencia de la interfaz visual, permitiendo modificar las consultas de Room o los esquemas de base de datos sin alterar los componentes gráficos de Jetpack Compose.

1. Diagrama del Modelo de Datos Relacional con Room

Para satisfacer los requerimientos del campus universitario y modelar el flujo de información de las edificaciones, se diseñó un modelo de datos relacional jerárquico compuesto por tres entidades vinculadas mediante relaciones de uno a muchos (1:N):

1. **`EdificioEntity`:** Representa el nivel superior de la infraestructura física del campus (por ejemplo, “Pabellón EPIS”, “Biblioteca Central”, “Auditorio Magno”).
2. **`AmbienteEntity`:** Representa los recintos, laboratorios o salas ubicados dentro de un edificio específico (por ejemplo, “Lab 101 - Redes y Conectividad”, “Hemeroteca y Tesis”). Se vincula con `EdificioEntity` mediante la clave foránea `edificioId`.
3. **`NotaEntity`:** Modela las observaciones, apuntes técnicos, reportes de mantenimiento o estados de uso asociados a un ambiente determinado (por ejemplo, “Racks y switches de laboratorio operativos”, “Proyector láser y microfonía inalámbrica verificados”). Se vincula con `AmbienteEntity` mediante la clave foránea `ambienteId`.

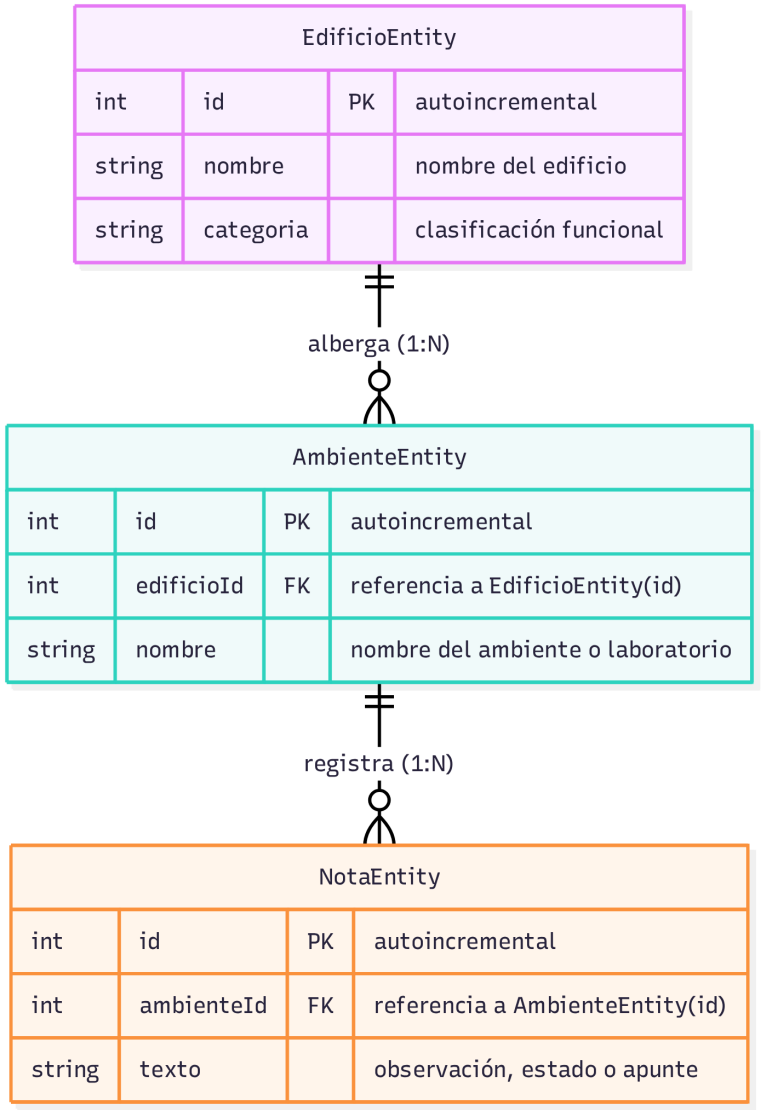


Figura 1: Diagrama Entidad-Relación (ERD) del modelo de datos con las tres entidades relacionadas en Room [1]

La cardinalidad del modelo establece que:

- Un **Edificio** puede albergar cero, uno o múltiples **Ambientes** ($1:N$). Si un edificio se elimina, sus ambientes asociados se eliminan en cascada (`ForeignKey.CASCADE`) para preservar la coherencia de la base de datos.
- Un **Ambiente** pertenece a un único **Edificio** y puede registrar cero, una o múltiples **Notas** ($1:N$). Al eliminarse un ambiente, todas sus notas vinculadas se eliminan automáticamente mediante la misma regla de propagación en cascada.

2. Implementación de Entidades, Claves Foráneas y DAOs

La implementación técnica de las entidades en Room se realizó mediante clases de datos Kotlin anotadas con `@Entity`. En cumplimiento de las buenas prácticas de ingeniería de bases de datos relacionales, se definieron restricciones de

clave foránea (`ForeignKey`) con acción de borrado en cascada (`onDelete = ForeignKey.CASCADE`) y se indexaron explícitamente las columnas foráneas (`indices = [Index(value = ["..."])]`) para prevenir escaneos de tabla completa (*full table scans*) en SQLite durante las operaciones de búsqueda y eliminación:

```
@Entity(
    foreignKeys = [
        ForeignKey(
            entity = AmbienteEntity::class,
            parentColumns = ["id"],
            childColumns = ["ambienteId"],
            onDelete = ForeignKey.CASCADE
        )
    ],
    indices = [Index(value = ["ambienteId"])]
)
data class NotaEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val ambienteId: Int,
    val texto: String
)
```

Para la manipulación de los datos se construyó la interfaz `EdificioDao`, declarando operaciones atómicas y reactivas ejecutadas en hilos de fondo mediante funciones de suspensión (`suspend`) de coroutines de Kotlin:

- **Inserción:** `@Insert suspend fun insertarEdificio(...)`, `insertarAmbiente(...)`, `insertarNota(...)`.
- **Actualización:** `@Update suspend fun actualizarEdificio(...)`, `actualizarAmbiente(...)`, `actualizarNota(...)`.
- **Eliminación:** `@Delete suspend fun eliminarEdificio(...)`, `eliminarAmbiente(...)`, `eliminarNota(...)`.
- **Lectura:** Consultas directas `@Query` y consultas relacionales transaccionales `@Transaction`.

La base de datos se encapsuló en `AppDatabase`, configurada como un singleton seguro frente a hilos (*thread-safe*) mediante `@Volatile` y doble comprobación de bloqueo (*double-checked locking*):

```
@Database(
    entities = [EdificioEntity::class, AmbienteEntity::class, NotaEntity::class],
    version = 1,
    exportSchema = false
)
abstract class AppDatabase : RoomDatabase() {
    abstract fun edificioDao(): EdificioDao

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        fun getInstance(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "app.db"
                ).fallbackToDestructiveMigration(true).build()
                INSTANCE = instance
                instance
            }
        }
    }
}
```

Para comprobar fehacientemente la creación física del archivo SQLite y la persistencia de las tres tablas, se inspeccionó el entorno mediante la herramienta *App Inspection / Database Inspector* de Android Studio conectada al emulador en ejecución:

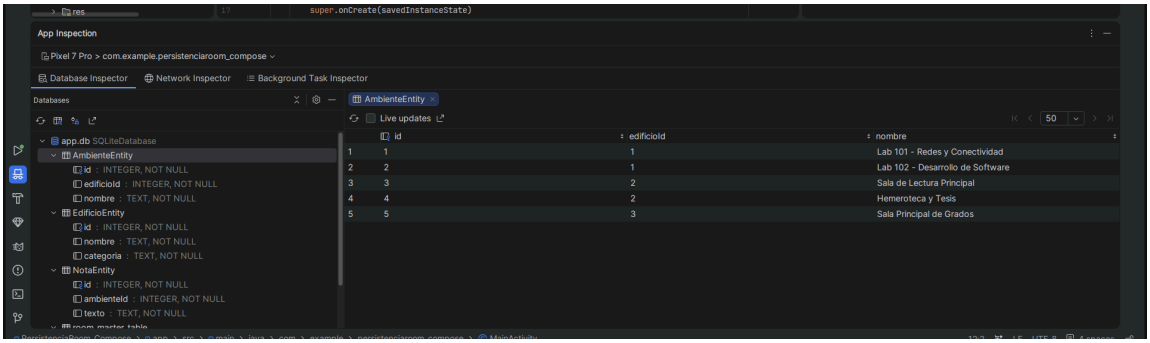


Figura 2: Visualización de la base de datos `app.db` en el Database Inspector de Android Studio: Tablas `EdificioEntity`, `AmbienteEntity`, `NotaEntity` y registros de ambientes [2]

Asimismo, se extrajo la base de datos directamente del almacenamiento de la aplicación para validar los registros almacenados en cada una de las tablas mediante el motor de comandos `sqlite3`:

```
-- Consulta directa sobre la base de datos SQLite física (app.db):
SELECT * FROM EdificioEntity;
-- 1|Pabellón EPIS|Académico
-- 2|Biblioteca Central|Servicios
-- 3|Auditorio Magno|Institucional

SELECT * FROM AmbienteEntity;
-- 1|1|Lab 101 - Redes y Conectividad
-- 2|1|Lab 102 - Desarrollo de Software
-- 3|2|Sala de Lectura Principal
-- 4|2|Hemeroteca y Tesis
-- 5|3|Sala Principal de Grados

SELECT * FROM NotaEntity;
-- 1|1|Racks y switches de laboratorio operativos
-- 2|1|Capacidad física: 32 estaciones de trabajo
-- 3|2|Equipos configurados con Android Studio y Compose
-- 4|3|Horario ininterrumpido de 07:00 a 20:00
-- 5|4|Consulta de material bibliográfico de posgrado
-- 6|5|Proyector láser y microfonía inalámbrica verificados
```

3. Consultas Relacionales con `@Relation` y `@Transaction`

En Room, cuando se requiere obtener entidades vinculadas en una estructura jerárquica de objetos sin recurrir a complejas combinaciones manuales (`JOIN`) ni desnormalizar los datos, se emplean clases intermedias decoradas con `@Embedded` y `@Relation` [1].

Se implementaron dos consultas relacionales independientes para resolver las necesidades del sistema:

1. **Relación 1: `EdificioConAmbientes` (Edificio con sus Ambientes asociados):** Permite que cada edificio agrupe automáticamente su lista de ambientes:

```
data class EdificioConAmbientes(
    @Embedded val edificio: EdificioEntity,
    @Relation(
        parentColumn = "id",
        entityColumn = "edificioId"
```

```
    )
    val ambientes: List<AmbienteEntity>
)
```

2. **Relación 2: `AmbienteConNotas` (Ambiente con sus Notas y Observaciones asociadas):** Constituye la segunda consulta relacional solicitada en la guía, permitiendo inspeccionar cada recinto junto a todas sus observaciones técnicas:

```
data class AmbienteConNotas(
    @Embedded val ambiente: AmbienteEntity,
    @Relation(
        parentColumn = "id",
        entityColumn = "ambienteId"
    )
    val notas: List<NotaEntity>
)
```

En la interfaz `EdificioDao`, ambas operaciones se anotan con `@Transaction`:

```
@Transaction
@Query("SELECT * FROM EdificioEntity")
suspend fun obtenerEdificiosConAmbientes(): List<EdificioConAmbientes>

@Transaction
@Query("SELECT * FROM AmbienteEntity")
suspend fun obtenerAmbientesConNotas(): List<AmbienteConNotas>
```

La anotación `@Transaction` es mandatoria porque Room realiza internamente múltiples consultas SQL separadas (una para la entidad padre y otra para recuperar los registros hijos asociados mediante cláusulas `IN`). Ejecutar estas consultas dentro de una transacción garantiza la atomicidad y coherencia temporal de la lectura, evitando inconsistencias o condiciones de carrera si otra operación en segundo plano modifica las tablas entre ambas lecturas.

Las dos consultas relacionales se visualizan en la aplicación mediante pestañas dedicadas:



Figura 3: Pestaña 1: Consulta relacional `EdificioConAmbientes` mostrando cada edificio con sus ambientes asignados

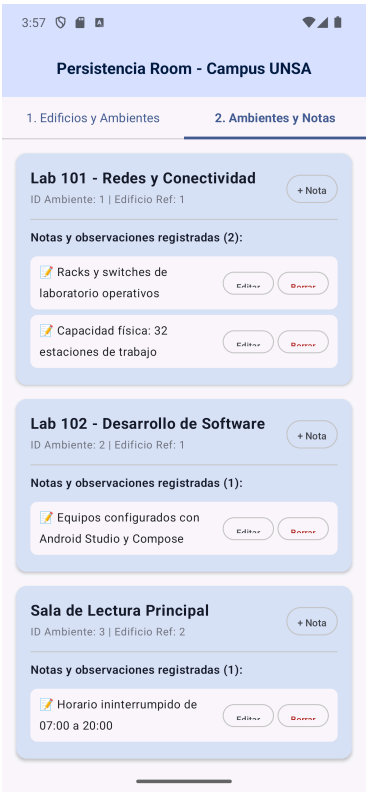


Figura 4: Pestaña 2: Consulta relacional `AmbienteConNotas` mostrando cada ambiente con sus observaciones técnicas

4. Interfaz de Usuario y Operaciones CRUD en Tiempo Real

Para ofrecer una experiencia interactiva completa, la interfaz gráfica implementada en `EdificiosScreen.kt` incluye controles visuales para ejecutar todas las operaciones de modificación y eliminación sobre los datos persistidos:

- **Navegación contextual por pestañas:** Se diseñó un encabezado interactivo con `TabRow` que permite alternar instantáneamente entre la vista de **Edificios y Ambientes** y la vista de **Ambientes y Notas**.
- **Botones de acción CRUD en cada elemento:**
  - Cada tarjeta de edificio dispone de botones para **Editar** su nombre/categoría y **Eliminar** el edificio completo (gatillando la eliminación en cascada de sus ambientes).
  - Cada ambiente incluye botones de **Editar** y **Borrar**, además de un botón rápido para agregar nuevos ambientes o notas.
  - Cada nota técnica dispone de botones dedicados para **Editar** su contenido textual y **Borrarla**.
- **Formularios en ventanas modales (`AlertDialog`):** Se construyó el módulo `DialogosCrud.kt` con tres diálogos modales (`DialogoEdificio`, `DialogoAmbiente`, `DialogoNota`) que validan los campos de entrada antes de despachar las coroutines de inserción o actualización hacia el `EdificiosViewModel`.
- **Botón de acción flotante (FAB):** Permite dar de alta nuevos edificios en cualquier momento.

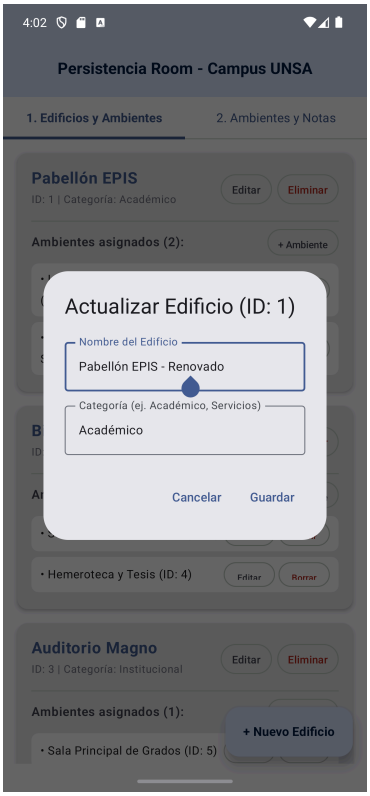


Figura 5: Diálogo modal interactivo para actualizar un edificio en tiempo de ejecución



Figura 6: Resultado de la actualización persistida (“Pabellón EPIS - Renovado”) reflejada reactivamente en la interfaz

Comprobación de la persistencia real: Tras editar el registro con identificador 1 a “Pabellón EPIS - Renovado” y guardar los cambios, la aplicación se cerró por completo (`adb shell am force-stop com.example.persistenciaroom_compose`) y se volvió a iniciar en frío. La consulta relacional cargó de inmediato los datos modificados directamente desde el archivo `app.db`, demostrando que los cambios son persistentes y no residen únicamente en la memoria volátil de la aplicación.

CUESTIONARIO

1. ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (datos de edificios y ambientes almacenados con Room, que persisten al cerrar y volver a abrir la app)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.

Sí, el Ejercicio Resuelto por el Docente se implementó y validó de forma íntegra. El sistema ejecuta con éxito el flujo completo de persistencia local mediante la arquitectura de tres componentes de Room (`Entity`, `DAO` y `Database`):

1. **Inicialización y Sembrado Inicial Condicional:** Al iniciar la aplicación por primera vez, el `EdificiosViewModel` consulta a través del repositorio si la tabla `EdificioEntity` se encuentra vacía (`dao.obtenerTodos().isEmpty()`). Al cumplirse la condición en una base de datos recién creada, se insertan dos edificios iniciales (“Biblioteca Central”

con categoría “Servicios”, y “Facultad de Ingeniería” con categoría “Académico”) junto con sus respectivos ambientes (“Sala de lectura”, “Hemeroteca”, “Laboratorio de cómputo”) utilizando claves foráneas que aseguran la integridad referencial.

2. **Consulta Relacional:** Se realiza la consulta `@Transaction @Query("SELECT * FROM EdificioEntity")` que recupera la estructura jerárquica `EdificioConAmbientes` mediante las anotaciones `@Embedded` y `@Relation`.
3. **Verificación de Persistencia tras Cierre Forzado:** Para verificar que los datos no residen únicamente en la memoria RAM volátil, la aplicación fue cerrada y expulsada de los procesos activos del sistema operativo mediante el comando:

   ```
   adb shell am force-stop com.example.persistenciaroom_compose
   ```

   Al reabrir la aplicación, la base de datos física `app.db` conservó intactos los registros previamente sembrados sin duplicar la información (evitado por la comprobación `isEmpty()`), cargando inmediatamente los registros en la lista `LazyColumn` de Jetpack Compose.



Figura 7: Evidencia de ejecución en el emulador: Persistencia de edificios y sus ambientes relacionados tras reinicio en frío de la aplicación

2. ¿Qué otra alternativa de almacenamiento (además de Room) se podría emplear en una aplicación con necesidades similares? Justifica tu respuesta comparando ventajas y desventajas frente a Room.

En el ecosistema de desarrollo de Android moderno existen múltiples alternativas a Room para persistir datos estructurados localmente. A continuación se analizan cuatro tecnologías comparables frente a las necesidades de modelado relacional de la aplicación:

A. SQLDelight

SQLDelight es una biblioteca desarrollada por Cash App basada en el paradigma *SQL-first*. A diferencia de Room (donde se escriben clases en Kotlin con anotaciones y la librería genera el SQL), en SQLDelight el desarrollador escribe sentencias SQL estándar en archivos `.sq`, y el compilador genera código Kotlin fuertemente tipado para interactuar con SQLite.

- **Ventajas frente a Room:**
  - **Soporte Kotlin Multiplatform (KMP):** Funciona de manera nativa en Android, iOS, Desktop (JVM) y Web (Wasm/JS), permitiendo compartir el 100% de la lógica de datos.
  - **SQL nativo sin magia de anotaciones:** Ofrece control absoluto sobre las consultas complejas, índices, disparadores y optimizaciones de SQLite.
- **Desventajas frente a Room:**
  - Curva de aprendizaje más pronunciada al requerir redacción manual exhaustiva de esquemas DDL y consultas DML.
  - Menor integración nativa con herramientas de Android Studio (por ejemplo, el Database Inspector tiene soporte de primer nivel optimizado para Room).

B. Realm / Atlas Device SDK

Realm es un motor de base de datos orientado a objetos y no relacional (NoSQL), implementado en C++ a nivel de núcleo, diseñado específicamente para dispositivos móviles.

- **Ventajas frente a Room:**
  - **Desaparición del desfase objeto-relacional (ORM):** Los objetos del modelo de dominio se guardan y leen directamente como objetos vivos en memoria, sin necesidad de mapeo entre tablas, columnas y filas de SQL.
  - **Sincronización en la nube integrada:** Cuenta con capacidades integradas para replicar datos automáticamente con bases de datos en la nube (MongoDB Atlas).
- **Desventajas frente a Room:**
  - Mayor sobrecarga en el tamaño final del archivo APK (alrededor de 3 a 5 MB adicionales debido a las librerías binarias nativas `.so`).
  - Bloqueo de proveedor (*vendor lock-in*) frente al formato universal y estándar de SQLite.

C. ObjectBox

ObjectBox es una base de datos orientada a objetos de altísimo rendimiento con soporte para relaciones entre entidades mediante enlaces de punteros en memoria (*Zero-Copy architecture*).

- **Ventajas frente a Room:**
  - Rendimiento en operaciones de lectura y escritura significativamente superior a SQLite/Room en escenarios de alto volumen de transacciones por segundo.
  - Definición directa de relaciones 1:N mediante clases `ToMany` y `ToOne` sin necesidad de claves foráneas relacionales.
- **Desventajas frente a Room:**
  - Ecosistema y comunidad más reducidos en comparación con la adopción masiva de Room respaldada oficialmente por Google.

D. DataStore (Preferences y Proto)

Jetpack DataStore es el reemplazo moderno de `SharedPreferences`. Utiliza coroutines de Kotlin y Flow para almacenar pares clave-valor (Preferences DataStore) o estructuras de datos serializadas con Protocol Buffers (Proto DataStore).

- **Ventajas frente a Room:**
  - Muy liviano y simple para almacenar configuraciones del usuario, tokens o estados atómicos de la interfaz.
- **Desventajas frente a Room:**
  - **Inadecuado para datos relacionales:** No soporta consultas indexadas, filtros condicionales complejos, ni relaciones jerárquicas (1:N) como Edificios → Ambientes → Notas. Toda modificación requiere reescribir el archivo completo en disco.

Tecnología	Paradigma	Validación en Compilación	Soporte KMP	Idoneidad Relacional
Room	Relacional (ORM / SQLite)	Sí (mediante KSP)	Parcial (Room 2.7+)	Excelente
SQLDelight	Relacional (SQL-first)	Sí (desde archivos `.sq`)	Total (Android, iOS, JVM)	Excelente
Realm	NoSQL (Objetos vivos)	Sí (esquema tipado)	Limitado a Kotlin SDK	Media (NoSQL)
ObjectBox	NoSQL (Objetos / Enlaces)	Sí (procesador de código)	Limitado	Media (NoSQL)
DataStore	Clave-Valor / Protobuf	Solo con Protobuf	Total	No idóneo

Conclusión comparativa: Para una aplicación nativa de Android con requerimientos de relaciones estructuradas (Edificios → Ambientes → Notas) e integración con Jetpack Compose, **Room** sigue siendo la opción estándar recomendada por su soporte oficial en la plataforma, verificación estricta de consultas SQL en tiempo de compilación mediante KSP y consumo eficiente de recursos del sistema.

3. ¿En dónde almacena Room los datos de la aplicación (almacenamiento interno o externo del dispositivo)? Justifica tu respuesta y explica qué implica para la privacidad de los datos del usuario.

Ubicación física del almacenamiento: Room almacena los datos de la base de datos de manera estricta y predeterminada en el **almacenamiento interno privado** (*Internal Storage*) del dispositivo Android.

Específicamente, el archivo `app.db` se aloja en el directorio del entorno de seguridad de la aplicación dentro de la partición `/data`:

```
/data/data/com.example.persistenciaroom_compose/databases/app.db
```

Cuando la base de datos opera bajo el modo WAL (*Write-Ahead Logging*) de SQLite, se generan dos archivos complementarios en la misma ruta:

- `app.db-wal`: Registro de transacciones previas a la sincronización en disco, optimizando la concurrencia entre lecturas y escrituras simultáneas.
- `app.db-shm`: Archivo de memoria compartida (*Shared Memory*) que coordina los punteros del registro WAL entre distintos procesos e hilos.

Justificación técnica y aislamiento por Sandboxing: Esta ubicación se rige por el modelo de seguridad fundamental de Android basado en el *Sandboxing* del kernel de Linux:

1. **Aislamiento de Identidad (UID):** Durante la instalación del paquete `com.example.persistenciaroom_compose`, el sistema operativo le asigna un identificador de usuario único (`UID`, por ejemplo `u0_a185`).
2. **Permisos del Sistema de Archivos POSIX:** El directorio privado de la aplicación y sus archivos de base de datos se crean con permisos restringidos (`rwx------` para el directorio y `rw-------` para los archivos). Esto significa que **únicamente los procesos ejecutados bajo el UID de la aplicación** tienen privilegios para abrir, leer o escribir en esos ficheros.

3. **Inaccesibilidad Externa:** Ninguna otra aplicación instalada en el dispositivo (incluso si cuenta con permisos generales de almacenamiento como `READ_EXTERNAL_STORAGE` o `MANAGE_EXTERNAL_STORAGE`) puede acceder al directorio `/data/data/com.example.persistenciaroom_compose/`. Solo un usuario con privilegios de superusuario (*root*) o una sesión de depuración autorizada por ADB (`run-as`) puede inspeccionar dicho directorio.
4. **Ciclo de Vida Acoplado:** Si el usuario desinstala la aplicación o utiliza la opción “Borrar datos de almacenamiento” desde los ajustes del sistema, el sistema operativo purga automáticamente todo el árbol de directorios en `/data/data/`, garantizando que no queden restos de información en el almacenamiento persistente.

Implicaciones para la privacidad de los datos del usuario:

- **Protección contra fugas de información:** A diferencia del almacenamiento externo compartido (como `/sdcard/` o `Download/`), donde cualquier aplicación con permisos puede leer los ficheros de otras aplicaciones, el almacenamiento interno de Room garantiza la confidencialidad de la información (notas de mantenimiento, registros de infraestructura e inventarios de laboratorios).
- **Cumplimiento normativo:** Esta arquitectura garantiza el principio de privacidad por diseño (*Privacy by Design*) exigido por marcos regulatorios como el GDPR o la Ley de Protección de Datos Personales (LPDP), evitando que datos personales o institucionales queden expuestos a vectores de ataque locales.
- **Capacidad de cifrado avanzado:** Si una aplicación maneja datos altamente sensibles (médicos, bancarios o confidenciales), la capa de abstracción de Room permite incorporar la librería **SQLCipher** mediante `Room.databaseBuilder(...).openHelperFactory(SupportFactory(passphrase))`, logrando un cifrado transparente de 256 bits (AES-256) sobre el archivo físico en el almacenamiento interno para proteger los datos incluso frente a dispositivos vulnerados o con acceso root.

CONCLUSIONES

- **Validación estricta y seguridad de tipos en tiempo de compilación:** La integración de Room a través de KSP (*Kotlin Symbol Processing*) frente a SQLite tradicional (`SQLiteOpenHelper`) erradica por completo los fallos en tiempo de ejecución derivados de errores de sintaxis en sentencias SQL o desajustes entre tipos de datos de las columnas y las clases del modelo. Al verificar la validez de las consultas SQL, las anotaciones y los esquemas relacionales durante el proceso de compilación, Room proporciona una garantía de robustez de nivel empresarial indispensable para aplicaciones de misión crítica.
- **Modelado relacional y transaccionalidad atómica con `@Relation`:** La estrategia de modelado relacional empleada mediante las clases `EdificioConAmbientes` y `AmbienteConNotas` demostró la capacidad de Room para resolver relaciones jerárquicas complejas ($1:N$) sin necesidad de escribir complejas consultas con sentencias `JOIN` manuales. La obligatoriedad de la anotación `@Transaction` en los métodos del DAO asegura la atomicidad en la recuperación multiconsulta, evitando lecturas sucias o inconsistencias temporales en entornos concurrentes y reactivos.
- **Arquitectura desacoplada y flujo de datos unidireccional (UDF):** La estructuración del sistema mediante el patrón MVVM y la capa de Repositorio consolidó una nítida separación de responsabilidades. Al abstraer el acceso a datos detrás de `EdificioRepository` y exponer flujos observables (`StateFlow`) desde el `EdificiosViewModel`, los componentes gráficos de Jetpack Compose (`LazyColumn`, `TabRow`, `AlertDialog`) operan como consumidores puros de estado, facilitando el mantenimiento, la modularidad y la futura implementación de pruebas unitarias o de instrumentación.
- **Integridad referencial y optimización de índices en SQLite:** La configuración explícita de claves foráneas con reglas de eliminación en cascada (`ForeignKey.CASCADE`) garantiza la integridad estructural de la base de datos, impidiendo la existencia de registros huérfanos cuando se elimina una entidad contenedora (edificio o ambiente). Asimismo, la creación de índices específicos sobre las columnas de clave foránea optimiza los tiempos de respuesta del motor SQLite al evitar escaneos de tabla completa (*full table scans*) en operaciones de búsqueda y borrado masivo.
- **Seguridad por diseño mediante almacenamiento interno y Sandboxing:** La persistencia física en el almacenamiento interno privado (`/data/data/<paquete>/databases/app.db`) respaldada por el modelo de permisos POSIX y el *Sandboxing* del kernel de Android garantiza el aislamiento total de los datos frente a accesos no autorizados de otras aplicaciones instaladas en el dispositivo, asegurando el cumplimiento de los estándares internacionales de privacidad y protección de datos del usuario.

REFERENCIAS

Bibliography

[1] Google Developers, “Define relationships between objects in Room.” Accessed: Oct. 27, 2026. [Online]. Available: https://developer.android.com/training/data-storage/room/relationships

[2] Google Developers, “Save data in a local database using Room.” Accessed: Oct. 27, 2026. [Online]. Available: https://developer.android.com/training/data-storage/room